

AGENT EVAL LAB — Study Guide & Reference

Everything We Built Today

PART 1: AI Industry Context

TIME Magazine's Dual-Layer Web

What they do: Serve two completely different versions of the same website based on **User-Agent** header.

Who Asks	What They Get
Chrome, Safari, Firefox	Full HTML (303KB), design, images, human ads
Googlebot	Same full HTML (protects SEO rankings)
ClaudeBot, PerplexityBot, OAI-SearchBot	Stripped markdown (13KB) with AI-targeted ads baked in
GPTBot, ChatGPT-User	406 Blocked —straight denied

The ad layer: Vendor is **Mobian**. They format sponsor content as markdown Q&As that get fed into AI model context windows. Metrics are measured in **tokens**, not human eyeballs.

Key insight: Advertisers know bots are reading the ads. They are paying for **retrieval influence** — the chance their brand gets mentioned when a user asks an AI “what’s the best online bank?”

Can a bot detect the ads? Yes, trivially. They are explicitly labeled > **Sponsored content**. The hidden part is the **audience split** — humans have no idea this parallel layer exists.

Nvidia's \$750B Bet

The paradox: Nvidia is financing the same customers who are building chips to replace Nvidia.

Why it makes sense: Nvidia owns the **training** market. They don't care which AI lab wins, as long as everyone buys their GPUs to get there.

Where they are vulnerable: Inference — running trained models for user queries. That is where Google (TPU), Amazon (Trainium/Inferentia), Meta (MTIA), and OpenAI (custom chips) are all building alternatives.

The inference commoditization threat: As inference gets cheaper, the “middle layer” of AI work dies fast:

- Generic prompt engineering
- AI content farms
- Junior devs who only glue APIs together
- Data entry and basic analysis

Michael Burry's short thesis: Market is pricing in perfection forever. Concentration risk + customers becoming competitors + potential overcapacity = bubble behavior.

Skills That Survive (The 3 Buckets)

Bucket 1: The “Plumber” Layer Infrastructure, orchestration, loop engineering. Wiring AI into real systems.

Skill	Why It Matters
LangGraph / agent orchestration	Building systems where agents talk, use tools, recover from errors
State management & memory	Agents that remember context across long workflows
Tool use & API integration	Connecting agents to real databases, CRMs, Slack, email
Observability & evals	Testing, monitoring, guardrails —the trust layer
Retrieval systems (RAG)	Feeding the right context so the model answers accurately

Bucket 2: The “Translator” Layer Explaining AI to non-technical decision-makers. Technical translation with business context.

- Cost modeling for AI deployments
- Regulatory awareness (EU AI Act, US state laws, China’s AI governance)
- Vendor evaluation (OpenAI vs Anthropic vs local models)

Bucket 3: The “Builder” Layer Building products end-to-end — identifying real problems and shipping working tools.

PART 2: The Agent Eval Lab Project

What We Built

A local agent that:

1. Takes a natural language question about GitHub repos
2. Decides which tool to call (get_commits or get_issues)
3. Hits the real GitHub API
4. Synthesizes a human-readable answer
5. Runs through 5 structured eval tests
6. Logs every step to LangSmith for tracing

All running locally on an 8GB GPU.

Folder Structure

```
AGENT EVAL LAB/  
├─ venv/                # Python virtual environment  
├─ .env                 # API keys (LangSmith)
```

```
├─ github_agent.py          # The agent + tools
├─ run_evals.py            # Eval runner + scoring
├─ langsmith_setup.py      # Tracing configuration
├─ evals/
│   └─ github_eval_dataset.json # 5 test cases
```

File 1: .env

```
LANGCHAIN_API_KEY=your_langsmith_key_here
LANGCHAIN_PROJECT=agent-eval-lab
```

Get your key at: <https://smith.langchain.com> (free tier)

File 2: langsmith_setup.py

```
import os
from dotenv import load_dotenv

load_dotenv()

def setup_tracing():
    os.environ.setdefault("LANGCHAIN_TRACING_V2", "true")
    os.environ.setdefault("LANGCHAIN_ENDPOINT", "https://api.smith.langchain.com")
    print(f"Tracing enabled for project: {os.environ.get('LANGCHAIN_PROJECT', 'default')}")
```

File 3: github_agent.py

```
import requests
from langchain_core.tools import tool
from langchain_ollama import ChatOllama
from langgraph.prebuilt import create_react_agent

@tool
def get_commits(repo: str, limit: int = 5) -> str:
    url = f"https://api.github.com/repos/{repo}/commits?per_page={limit}"
    response = requests.get(url, timeout=15)
    if response.status_code != 200:
        return f"Error: GitHub returned {response.status_code} for repo '{repo}'"
    commits = response.json()
    lines = []
    for c in commits:
        sha = c.get("sha", "?")[:7]
        author = c.get("commit", {}).get("author", {}).get("name", "Unknown")
        msg = c.get("commit", {}).get("message", "No message").split("\n")[0][:60]
        lines.append(f"- {sha} by {author}: {msg}")
    return "\n".join(lines) if lines else "No commits found."
```

```

@tool
def get_issues(repo: str, label: str = "bug", limit: int = 5) -> str:
    url = f"https://api.github.com/repos/{repo}/issues?state=open&labels={label}&per_page={limit}"
    response = requests.get(url, timeout=15)
    if response.status_code != 200:
        return f"Error: GitHub returned {response.status_code} for repo '{repo}'"
    issues = response.json()
    if not issues:
        return f"No open issues with label '{label}' found in {repo}."
    lines = []
    for i in issues:
        num = i.get("number", "?")
        title = i.get("title", "No title")
        user = i.get("user", {}).get("login", "Unknown")
        lines.append(f"- #{num} {title} (by {user})")
    return "\n".join(lines)

model = ChatOllama(
    model="llama3.1:8b",
    temperature=0,
    base_url="http://127.0.0.1:11434"
)

tools = [get_commits, get_issues]
agent = create_react_agent(model, tools)

def run_agent(query: str):
    result = agent.invoke({"messages": [{"user", query}]})
    return result

```

File 4: evals/github_eval_dataset.json

```

[
  {
    "name": "list_commits_langchain",
    "input": "What were the last 3 commits to the langchain-ai/langchain repo?",
    "expected_tools": ["get_commits"],
    "expected_in_output": ["by", "commit"]
  },
  {
    "name": "list_bugs_react",
    "input": "Are there any open bugs in facebook/react?",
    "expected_tools": ["get_issues"],
    "expected_in_output": ["#", "by"]
  },
  {
    "name": "commits_and_issues",
    "input": "Show me recent commits and any bug issues for torvalds/linux",
    "expected_tools": ["get_commits", "get_issues"],

```

```

    "expected_in_output": ["by", "#"]
  },
  {
    "name": "nonexistent_repo",
    "input": "What are the commits to this-is-fake-repo-12345/nowhere?",
    "expected_tools": ["get_commits"],
    "expected_in_output": ["Error", "404"]
  },
  {
    "name": "repo_with_no_bugs",
    "input": "Find bug issues in octocat/Hello-World",
    "expected_tools": ["get_issues"],
    "expected_in_output": ["No open issues", "found"]
  }
]

```

File 5: run_evals.py

```

import json
import os
from github_agent import run_agent
from langsmith_setup import setup_tracing

setup_tracing()

def load_eval_dataset(path: str) -> list:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def score_run(test_case: dict, agent_result: dict) -> dict:
    messages = agent_result.get("messages", [])
    if not messages:
        return {"test": test_case["name"], "score": 0, "max": 3, "passed": False, "error": "No messages
        ↳ returned"}

    final_content = messages[-1].content or ""

    tools_called = []
    for msg in messages:
        if hasattr(msg, "tool_calls") and msg.tool_calls:
            for tc in msg.tool_calls:
                if isinstance(tc, dict):
                    tools_called.append(tc.get("name", "unknown"))
                else:
                    tools_called.append(getattr(tc, "name", "unknown"))

    expected_tools = test_case.get("expected_tools", [])
    tools_matched = any(t in tools_called for t in expected_tools)

```

```

expected_outputs = test_case.get("expected_in_output", [])
output_hits = sum(1 for exp in expected_outputs if exp.lower() in final_content.lower())
output_score = output_hits / max(len(expected_outputs), 1)

no_crash = "error: github returned" not in final_content.lower() or "404" in final_content.lower()

score = (1.0 if tools_matched else 0.0) + output_score + (1.0 if no_crash else 0.0)

return {
    "test": test_case["name"],
    "score": round(score, 2),
    "max": 3,
    "tools_called": tools_called,
    "passed": score >= 2.0,
    "snippet": final_content[:200].replace("\n", " ")
}

def main():
    dataset = load_eval_dataset("evals/github_eval_dataset.json")
    results = []

    print(f"Running {len(dataset)} eval cases...\n")

    for case in dataset:
        print(f"Test: {case['name']}: {case['input'][:60]}...")
        result = run_agent(case["input"])
        score = score_run(case, result)
        results.append(score)

        status = "PASS" if score["passed"] else "FAIL"
        print(f"    {status} | Score: {score['score']}/3 | Tools: {score['tools_called']}")
        print(f"    Output: {score['snippet'][:100]}...")
        print()

    total = sum(r["score"] for r in results)
    max_total = sum(r["max"] for r in results)
    passed = sum(1 for r in results if r["passed"])

    print("=" * 50)
    print(f"FINAL: {total}/{max_total} ({total/max_total*100:.1f}%)")
    print(f"Passed: {passed}/{len(results)} cases")

if __name__ == "__main__":
    main()

```

PART 3: Setup & Troubleshooting

Step-by-Step Setup

1. **Create folder** AGENT EVAL LAB on Desktop
2. **Open terminal** in that folder
3. **Create virtual environment:**

```
python -m venv venv
```

4. **Activate it:**

```
venv\Scripts\activate
```

Look for (venv) in your prompt

5. **Install packages:**

```
pip install langgraph langchain langsmith requests python-dotenv langchain-ollama
```

6. **Pull the model:**

```
ollama pull llama3.1:8b
```

7. **Verify Ollama is running:**

```
curl http://localhost:11434
```

Or browser: <http://localhost:11434> — should say “Ollama is running”

8. **Create all 5 files** listed above
9. **Add your LangSmith API key** to .env
10. **Run the evals:** `bash` `python run_evals.py`

Common Errors & Fixes

Error	Cause	Fix
ModuleNotFoundError: langchain_ollama	Forgot to install	<code>pip install langchain-ollama</code>
WinError 10049 / ConnectError	localhost resolves to IPv6 on Windows	Add <code>base_url="http://127.0.0.1:11434"</code> to ChatOllama
Error: model not found Connection refused	Ollama doesn't have the model Ollama isn't running	<code>ollama pull llama3.1:8b</code> Check system tray for llama icon; start Ollama app
Error: GitHub returned 403 LANGCHAIN_API_KEY not set	Rate limited by GitHub .env file missing or wrong	Wait 60 seconds and retry Check .env exists in root folder with correct key

PART 4: Reading LangSmith Traces

What You Are Looking At

Traces list (the rows): Each row = one complete agent run (one eval test case).

Flowchart view (click a row): Shows the step-by-step execution:

```
LangGraph (total time, total tokens)
├─ agent
│   └─ call_model
│       ├── Prompt (system instructions + user question)
│       └─ ChatOllama (the model "thinking" — deciding which tool to use)
│   └─ should_continue ("do I need another tool or am I done?")
├─ tools
│   └─ get_commits / get_issues (actual API call to GitHub)
└─ agent (second pass — synthesizes final answer from tool results)
```

Key Metrics

Metric	What It Means
Time per node	How long that step took. Long <code>call_model</code> = model is “thinking” hard. Long <code>tools</code> = API is slow or returned lots of data.
Tokens	How much text the model processed. Think “word count.”
Green check	Trace completed successfully.
Red X	Trace failed —click it to see the error.

Why Times Vary

- **Small repo + no issues** = fast (3-4s)
- **Huge repo** (torvalds/linux) = slower API response + more data to parse
- **Model hesitation** = longer `call_model` time as the 8B model reasons through the decision

PART 5: Eval Datasets — The Concept

What Is an Eval Dataset?

A collection of **test cases** where each case has:

- **Input:** What the user asked
- **Expected behavior:** What the agent *should* do (which tools to call)
- **Expected output:** What the final answer should contain

NOT for training the model. For **testing** whether your agent works reliably.

The Scoring Rubric (3 Points Per Test)

Criterion	Points	How It Is Scored
Tool selection	1	Did it call the right tool?
Output quality	1	Does output contain expected keywords?
No crash	1	Did it complete without unhandled errors?

Pass threshold: 2.0/3 or higher = “good enough to ship”

How to Build Your Own Evals

1. List what your agent is supposed to do
2. Create 10-20 test cases per task:
 - **Happy path** (everything works)
 - **Edge case** (empty data, weird input)
 - **Adversarial case** (off-topic question, API timeout)
3. Run all cases, score them, establish a baseline
4. Change your agent, re-run, see if score improves

PART 6: Free Data Sources for Practice

Source	URL	Good For
GitHub API	api.github.com	Tool-use agents, commit/issue tracking
Hacker News API	github.com/HackerNews/API	Summarization + filtering agents
Reddit JSON	reddit.com/r/LocalLLaMA.json	Content analysis
HuggingFace Datasets	huggingface.co/datasets	Pre-built eval benchmarks
Gorilla Function Calling	gorilla-llm/ function-calling-eval	Testing tool selection accuracy
AgentBench	github.com/THUDM/AgentBench	Multi-step agent tasks
WebShop	github.com/princeton-nlp/ WebShop	Simulated e-commerce browsing

PART 7: Key Takeaways

1. **The web is splitting in two** — human-facing and machine-facing layers. The machine layer is where persuasion is happening now.
2. **Nvidia is paying its own assassins** — financing customers who are building chips to replace them. Inference commoditization is the real threat.
3. **The jobs that survive** are plumbers (orchestration), translators (business context), and builders (end-to-end products). Not prompt engineers or API gluers.

4. **Evals and tracing are the discipline** that turns a demo into a product. Anyone can make an agent work once. The pros can prove it works reliably.
5. **Your 8GB card is a feature, not a bug.** It forces you to focus on agent logic (orchestration, memory, tool use) instead of just throwing bigger models at problems.
6. **Ship something real in 3-4 months.** Not more tutorials. Build, break, fix, repeat. Your runway is ticking.

Generated: August 5, 2026 For: Agent Eval Lab project & AI infrastructure study